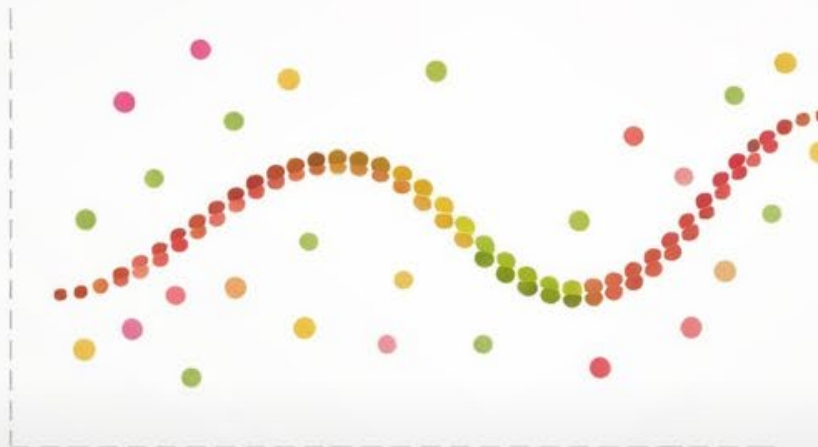
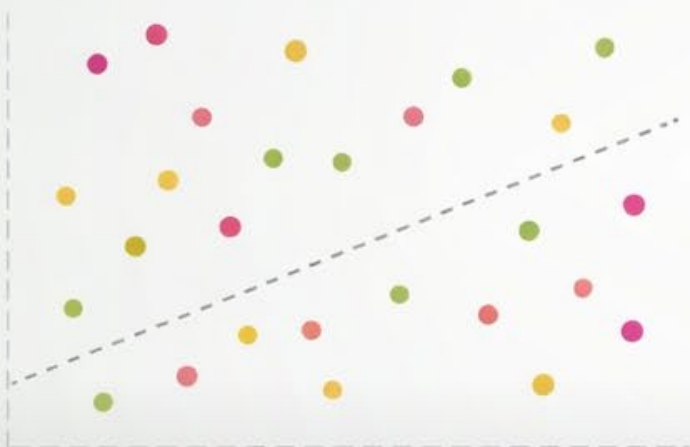
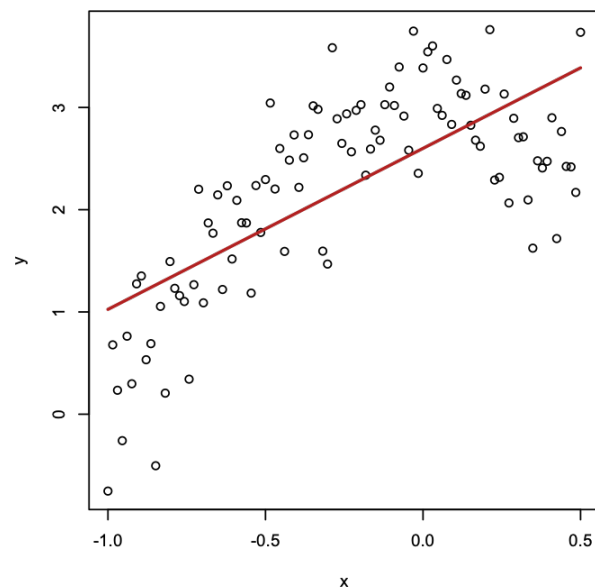




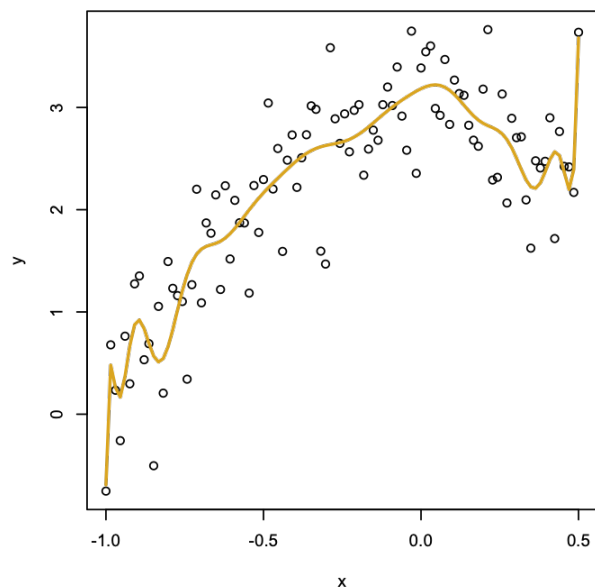
MODEL FITTING

OVERFITTING VS UNDERFITTING

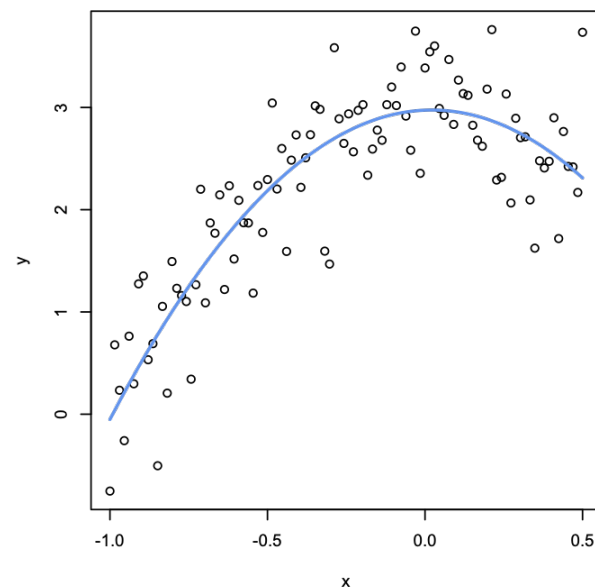
Underfit Model



Overfit Model



Good Model



UNDERFITTING

- Symptoms, causes, and fixes for a model that is too simple
 - Identifying: The model performs poorly on both training and test data
 - Possible fixes:
 - Use a more complex model
 - Find better features
 - Reduce constraints on model (less regularization)
 - Tune hyperparameters
-

OVERFITTING

- Symptoms, causes, and fixes for a model that is too complex
 - Identifying: The model performs well on training data but poorly on test data
 - Possible fixes:
 - Use a simpler model
 - Add constraints on model (more regularization)
 - Gather more data
 - Tune hyperparameters
-

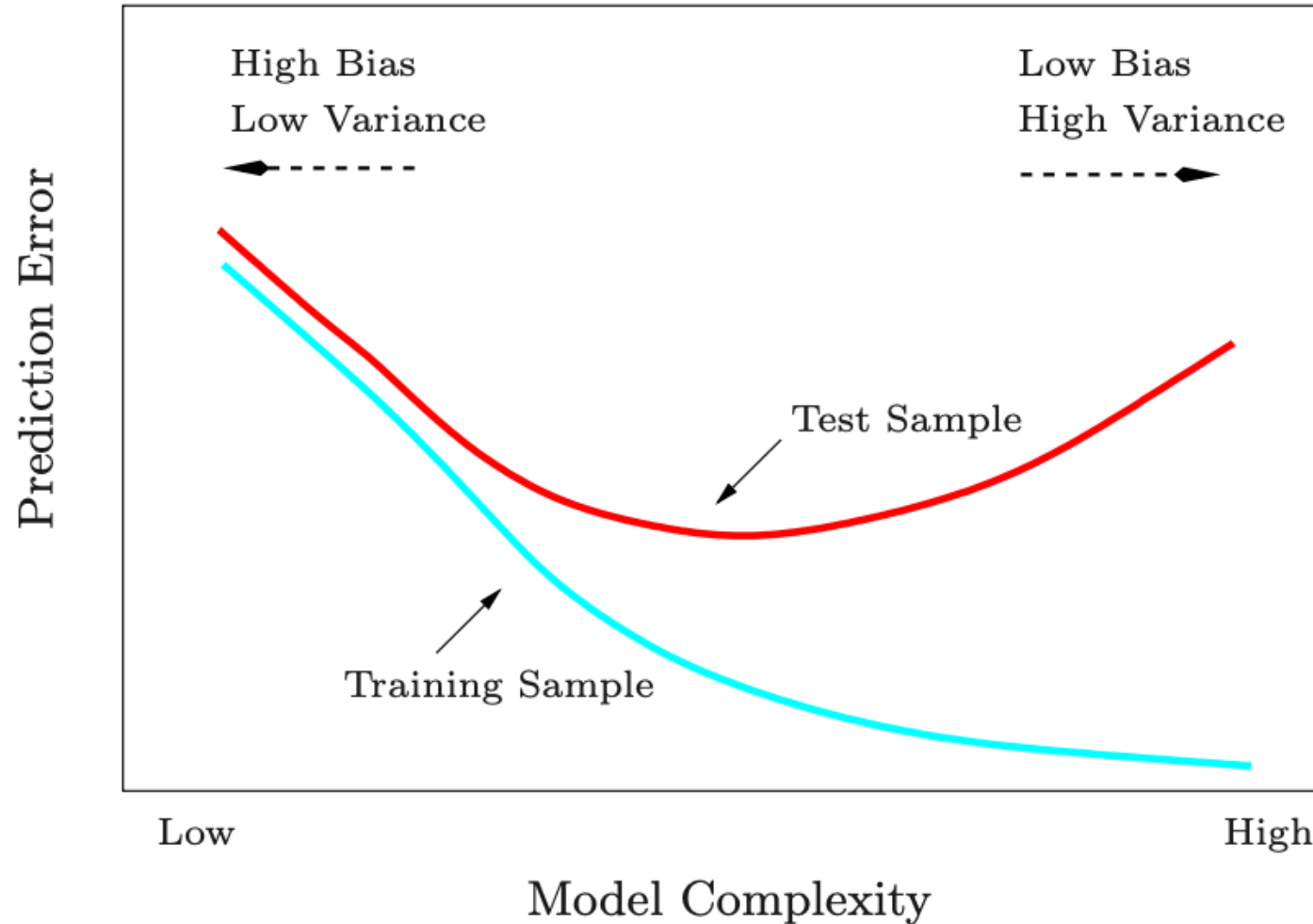
CHECKPOINT

- A model reports training $R^2 = 0.99$ and test $R^2 = 0.42$
 - What is happening, and what would you try first?
 - Which error is large — training, test, or both?
 - Does the gap point to high bias or high variance?
 - Would a simpler model, more regularization, or more data help?
 - Two minutes with a neighbor, then we compare answers
-

BIAS AND VARIANCE

- Assume the truth is $y = f(x) + \varepsilon$, with $E[\varepsilon] = 0$ and $Var(\varepsilon) = \sigma^2$
 - Fit an estimate $\hat{f}$ on a training set, then predict at one fixed point x_0
 - The two quantities we need:
 - $Bias(\hat{f}(x_0)) = E[\hat{f}(x_0)] - f(x_0)$:how far the average fit sits from the truth
 - $Var(\hat{f}(x_0)) = E[(\hat{f}(x_0) - E[\hat{f}(x_0)])^2]$:how much the fit moves across training sets
 - Every expectation is taken over random training sets and over the noise ε
-

BIAS-VARIANCE TRADEOFF



BIAS-VARIANCE TRADEOFF

- The balance of finding a model that doesn't underfit or overfit is related to the bias-variance tradeoff
 - **Bias** is the error from approximating a real-life problem with a simpler model
 - **Variance** is how much the prediction changes with a different training set from the same population
 - In general, as a model becomes more flexible and complex, variance increases and bias decreases
 - **Overfitting** is associated with high **variance**
 - **Underfitting** is associated with high **bias**
-

DERIVING THE TRADEOFF (PROOF SKETCH)

$$MSE = E[(y_0 - \hat{f}(x_0))^2] = E[(f(x_0) + \varepsilon - \hat{f}(x_0))^2]$$

substitute $y_0 = f(x_0) + \varepsilon$

$$= E[(f(x_0) - \hat{f}(x_0))^2] + \sigma^2$$

ε is independent with $E[\varepsilon] = 0$, $Var(\varepsilon) = \sigma^2$

$$= E[((f(x_0) - E[\hat{f}(x_0)]) + (E[\hat{f}(x_0)] - \hat{f}(x_0)))^2] + \sigma^2$$

add and subtract $E[\hat{f}(x_0)]$

$$= [Bias(\hat{f}(x_0))]^2 + Var(\hat{f}(x_0)) + \sigma^2$$

the cross term has expectation zero $\rightarrow$ bias² + variance + irreducible error

BIAS-VARIANCE TRADEOFF

$$\bullet E[(y_0 - \hat{f}(x_0))^2] = \underbrace{\text{Var}(\hat{f}(x_0))}_{\text{Variance}} + \underbrace{[\text{Bias}(\hat{f}(x_0))]^2}_{\text{Bias (squared)}} + \underbrace{E[(y - f(x))^2]}_{\text{Irreducible error}}$$

- Variance of $\hat{f}$ refers to the amount that $\hat{f}$ changes if a different training set is used (but from the same “population”)
 - Bias of $\hat{f}$ refers to the error that is introduced by approximating a real-life problem with a simpler model
 - Irreducible error: Random noise inherent in the data. The irreducible error is the best-case scenario given the noise in the data
 - Tradeoff because usually reducing bias will increase variance and visa versa
-

K-NN RECALL

- Recall what we learned about KNN classification and regression
 - Will a **low** value of k be more likely to overfit or underfit?
 - How does this relate to bias/variance?
 - Will a **high** value of k be more likely to overfit or underfit?
 - How does this relate to bias/variance?

Testing and Validating

HOW WELL DOES MY MODEL WORK?

- In the prediction setting:
 - **Model evaluation:** how well does our machine learning model generalize to **new data**?
 - Don't get caught in the weeds of the possible validation methods
 - Find an **honest metric to quantify future model performance**

HOW WELL DOES MY MODEL WORK?

- The only way to evaluate how well a model predicts is to **apply it to new data**
- Comparing test metrics to training metrics can also help identify overfitting
- What if new data (with labels) isn't readily available (and it usually isn't)

MODEL EVALUATION STRATEGIES

TRAIN / VALIDATION / TEST

- Split the data once, and give each piece a single job
- The test set is touched exactly once, at the very end
- The three roles:
 - Training set (~60%): fit the model parameters
 - Validation set (~20%): tune hyperparameters, compare models
 - Test set (~20%): estimate generalization error

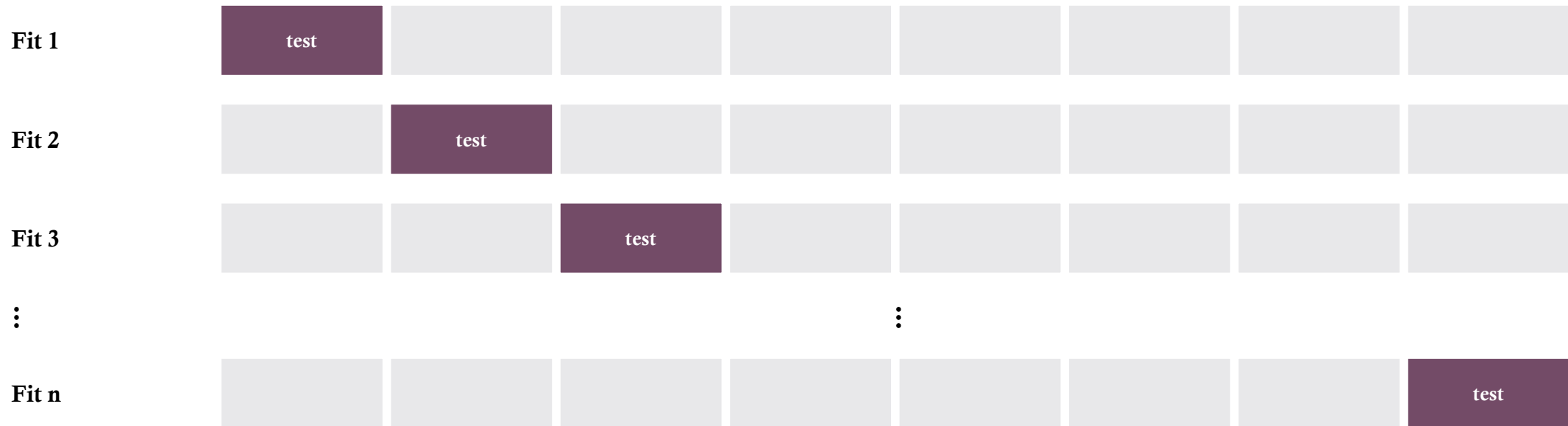
Training ~60%

Validation ~20%

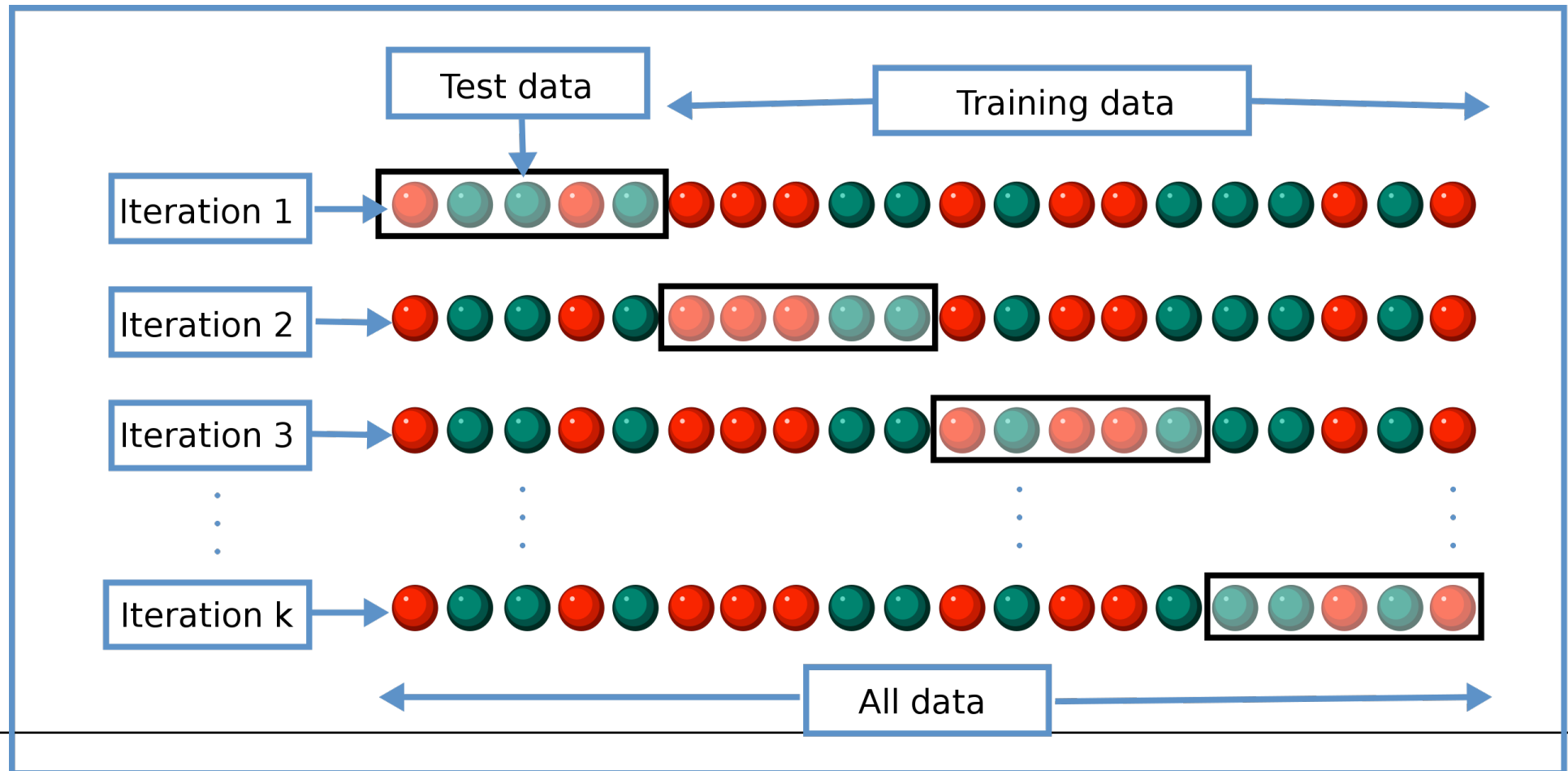
Test ~20%

LEAVE-ONE-OUT CROSS-VALIDATION (LOOCV)

- LOOCV is k-fold with $k = n$: every observation takes one turn as the test set
- The model is fit n times, each on the other $n - 1$ observations
- LOOCV is deterministic! (Not random).



K-FOLD CROSS-VALIDATION



CHOOSING K (THE NUMBER OF FOLDS)

- The data scientist has yet another choice to make: k
 - $k=n$ is leave-one-out cross-validation (LOOCV), this is deterministic
 - $k=5$ or $k=10$ are other popular choices
 - Bias–variance tradeoff in k -fold CV:
 - Small $k \rightarrow$ less training data per fit, so a more biased (pessimistic) estimate
 - Large $k \rightarrow$ nearly full training sets, so little bias, but overlapping folds make the estimate more variable
 - For tuning, bias matters less than finding the minimum error
 - Computational cost scales with k
 - LOOCV is by far the most expensive choice
-

HYPERPARAMETERS

- Also called tuning parameters
- Parameters of the learning algorithm and not the model
 - For example, k , in k-NN is a hyperparameter
- Hyperparameters are chosen by the modeler
 - Values that are actually *learned* from the data are usually called parameters

A TYPICAL APPROACH

- Split data into training and test sets
 - Train the model several times using different values of the hyperparameter
 - Choose the hyperparameter value that performs best on the training set as measured with k -fold cross-validation
 - Use the test set to compute the generalization error
 - Remember that the goal is to get a good estimate of the **generalization error**
-

A TEMPTING APPROACH...

- Split data into a training and a test set
 - Train the model for many hyperparameter values
 - Choose the hyperparameter value that performs best on test set
-

WHY THAT APPROACH LEAKS

- Choosing on the test set makes the test score optimistic
 - Illustrative: try 200 hyperparameter values, keep the best test score
 - What you see vs. what you get:
 - Reported test accuracy: 91%
 - Accuracy on genuinely new data: 84%
 - The 7-point gap is selection, not learning
 - Fix: tune with cross-validation inside the training set, or use nested CV
-

SUMMARY: FROM SYMPTOM TO REMEDY

- Diagnose from the gap between training and test error
 - Quick reference:
 - Both errors high $\rightarrow$ underfit $\rightarrow$ more flexibility, better features
 - Training low, test high $\rightarrow$ overfit $\rightarrow$ simplify, regularize, add data
 - Both low and close $\rightarrow$ good fit $\rightarrow$ report the test error once
 - Always keep an untouched estimate of generalization error
-